Arika Resort — Project Reference

What this project is

Arika Resort is a small **resort booking** web app: guests browse **rooms** and optional **add-on experiences**, then submit a **booking** with dates, guest count (1–2), and contact details. The UI is a **Next.js App Router** front end; **PostgreSQL on Supabase** holds catalog and booking data. Stays are modeled as **half-open date ranges** `[check_in, check_out)` (checkout day is exclusive), and the database enforces **no overlapping bookings per room**.

Tech stack

Layer	Choice
Framework	**Next.js 16** (App Router), **React 19**, **TypeScript**
Styling	**Tailwind CSS v4** (`@import "tailwindcss"` in `app/globals.css`, PostCSS plugin)
Fonts	**Geist** / **Geist Mono** via `next/font/google` in `app/layout.tsx`
Backend / DB	**Supabase** — `@supabase/ssr` for cookie-aware server/browser clients, `supabase-js` for a **service-role** server client
Validation	Pure TS in `lib/booking/` (dates, guest count); overlap check in `lib/supabase/queries/bookings.ts`

End-to-end flow

1. **Home** (`/`) — marketing copy and links to Rooms, Experiences, Book.
2. **Rooms** (`/rooms`) — server component loads `rooms` from Supabase, renders `RoomCard` grid. Each card links to detail and to `/book?roomId=…` .
3. **Room detail** (`/rooms/[roomId]`) — loads one room; invalid id → `notFound()` .
4. **Experiences** (`/experiences`) — lists `experiences` for context; booking form also loads these.
5. **Book** (`/book`) — loads rooms + experiences in parallel; renders client **`BookingForm`** with optional preselected room from `?roomId=` .
6. **Submit** — form posts to **Server Action** `createBookingAction` (`app/book/actions.ts`): validates input, checks **overlap** for that room/dates, inserts `booking` and optional `booking_experiences` rows using the **service role** client, then **redirects** to `/book/success?id=<uuid>` .
7. **Success** — static confirmation + reference id from query string.

Reads (lists/detail) use the **server Supabase client** tied to the request cookies (`utils/supabase/server.ts`) with the **publishable/anon** key — subject to **RLS** (select allowed on rooms/experiences).

Writes (booking) use **`createServiceRoleClient()`** (`utils/supabase/service.ts`), which **bypasses RLS**. That is why `SUPABASE_SERVICE_ROLE_KEY` must exist only on the server; the action validates input before inserting.

File-by-file reference

Root / config

File	Role

`package.json`	Scripts (`dev`, `build`, `start`, `lint`) and dependencies (Next, React, Supabase, Tailwind 4, ESLint).
`package-lock.json`	Locked dependency tree for reproducible installs.
`tsconfig.json`	TypeScript settings; `@/*` → project root imports.
`next.config.ts`	Next.js config (currently minimal placeholder).
`next-env.d.ts`	Auto-generated Next/TypeScript references (do not edit).
`eslint.config.mjs`	ESLint flat config using `eslint-config-next` (core-web-vitals + TypeScript).
`postcss.config.mjs`	PostCSS with `@tailwindcss/postcss` for Tailwind v4.
`README.md`	Default create-next-app starter text (not customized for this resort app).
`AGENTS.md`	Note for agents: this Next version may differ from older docs.
`CLAUDE.md`	Agent guidance for this repo.
`.env.local`	**Local secrets** (not committed): Supabase URL, publishable key, and **service role key** for server actions.
`test-supabase.js`	Small **Node script** using `dotenv` + Supabase client to `select` from `rooms` — handy to verify env and connectivity.

`app/` — routes and global UI

File	Role
`app/layout.tsx`	Root layout: HTML shell, header nav, Geist fonts, imports `globals.css`, sets metadata.
`app/globals.css`	Tailwind v4 entry + CSS variables / dark `prefers-color-scheme` for background/foreground.
`app/page.tsx`	Landing page with CTAs to `/rooms`, `/experiences`, `/book`.
`app/rooms/page.tsx`	Server page: `getRooms` → grid of `RoomCard`.
`app/rooms/[roomId]/page.tsx`	Dynamic room detail; uses `getRoomById`, handles errors and `notFound()`.
`app/experiences/page.tsx`	Server page: lists experiences via `getExperiences`.
`app/book/page.tsx`	Server page: parallel `getRooms` + `getExperiences`, passes `defaultRoomId` from `searchParams.roomId` into `BookingForm`.
`app/book/actions.ts`	`"use server"` — `createBookingAction`: validation, overlap check, insert booking + junction rows, `redirect` to success.
`app/book/success/page.tsx`	Thank-you page; shows booking id from `searchParams.id`.

Note: There is no `middleware.ts` at the project root — session refresh middleware is not wired; `utils/supabase/middleware.ts` is a standalone helper pattern.

`components/`

File	Role
`BookingForm.tsx`	**Client component** — `useActionState` wired to `createBookingAction`; full form fields matching server expectations; embeds `ExperienceSelector`.
`RoomCard.tsx`	Presentational card for a room with links to detail and pre-filled book URL.
`ExperienceSelector.tsx`	Checkbox group `name="experience_ids"` for optional add-ons.

`lib/`

File	Role
`lib/supabase/queries/rooms.ts`	`getRooms`, `getRoomById` — typed Supabase queries.
`lib/supabase/queries/bookings.ts`	`hasOverlappingBooking` (half-open overlap as SQL filters) and `getExperiences`.
`lib/booking/validation.ts`	ISO date strings, stay length, guest count 1–2.
`lib/booking/availability.ts`	Pure `stayRangesOverlap` helper (documents half-open semantics; overlap logic in DB query is the runtime path for booking).

`types/`

File	Role
`types/database.ts`	Hand-maintained **Supabase `Database` type** plus `RoomRow`, `ExperienceRow`, etc. Keeps inserts/selects type-safe.

`utils/supabase/`

File	Role
`server.ts`	`createServerClient` from `@supabase/ssr` with Next **cookies** — for Server Components / server-side reads.
`client.ts`	`createBrowserClient` — for future client-side Supabase usage.
`service.ts`	**Service role** client — server-only, no session persistence; used in `createBookingAction`.
`middleware.ts`	Pattern for Supabase + Edge `NextRequest` cookie refresh; not imported by a root `middleware.ts` in this project.

`supabase/`

File	Role
`migrations/20260419000000_resort_booking.sql`	Creates **`btree_gist`**, tables `rooms`, `experiences`, `booking`, `booking_experiences`; **CHECK** constraints; **EXCLUDE** constraint so the same room cannot have overlapping `[check_in, check_out)` ranges; enables

	RLS and policies for public read + anon/authenticated insert on booking tables; grants.
`migrations/20260420000000_fix_booking_rls_policies.sql`	Recreates insert policies in a form that avoids parser issues; re-grants `INSERT` for `anon`/`authenticated`.
`seed.sql`	Idempotent sample **rooms** and **experiences** UUID rows for local/dev testing.

Database model (conceptual)

- **`rooms`** — catalog (name, optional slug/description, `max_guests`).
- **`experiences`** — optional add-ons.
- **`booking`** — one row per reservation: `room_id`, `check_in` / `check_out` **dates**, guest count, contact fields.
- **`booking_experiences`** — many-to-many: which experiences were chosen for a booking.

Integrity:

- App + SQL **CHECK** align with 1–2 guests and `check_out > check_in`.
- **EXCLUDE … daterange `[)`** prevents double-booking the same room at the DB level (stronger than only app-level checks).
- **RLS:** anonymous/authenticated users can **SELECT** rooms/experiences; insert policies exist for booking tables (the app currently inserts via **service role**, which bypasses RLS).

Environment variables (from code expectations)

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` (anon/publishable client)
- `SUPABASE_SERVICE_ROLE_KEY` — **server only**, required for `createBookingAction` when using `createServiceRoleClient()`

Summary

This repo is a **focused resort booking demo**: **Next.js server-rendered pages** pull catalog data with the **SSR Supabase client**; **one client form** submits through a **Server Action** that validates, checks overlaps, and writes with the **service role** client, while **migrations** define **Postgres constraints and RLS** suitable for Supabase-hosted data.